

Task-Aware Object Detection (TAOD) Accelerator

Hardware Selection / Scoring Engine

DVCon India 2026 Design Contest

Stage 2B Report

Target platform: VEGA AS1061 (RV64IMAFDC) on Genesys-2 (Xilinx XC7K325T), 50 MHz, AXI4

Team: Ashes

Date: 30/06/2026

1. Introduction

This report documents Stage 2B of the EVIOS task aware object detection project: the RTL design, verification, and results for a hardware accelerator that performs task aware object detection.

Conventional object detection identifies the object without knowing the task, it locates all objects in a scene without regard to what the user actually wants to do. The contest problem asks for a task aware framework that, given a task description, selects (filters or prioritizes) the object most relevant to that task. Following the reference work (“What Object Should I Use? — Task Driven Object Detection”), detection is performed by an off-the-shelf model, and the task-aware step selects among the detected objects.

The accelerator presented here implements that selection step in hardware. It does not process pixels or perform detection; detection (a YOLO ensemble) runs upstream in software and produces a list of objects. For each object the accelerator receives its class, two model confidences (from the ensemble), bounding box, and a track identifier, together with a task identifier. It computes a per-object relevance score and selects the best object for the task.

Score per object:

$$score = F_{conf} \times w_{task} \times T_{stable} \times areaTerm$$

Where,

$$F_{conf} = w1 \cdot c1 + w2 \cdot c2$$

- is the ensemble-fused confidence.
- w_{task} is a per - (task, class) relevance weight read from an on-chip lookup table.
- T_{stable} is a per-track temporal-stability term maintained as an exponential moving average (EMA).
- $areaTerm$ is derived from the bounding-box area.

The engine then performs an argmax to select the highest-scoring object and tracks the runner-up.

2. Design Block Diagram

Figure 1 shows the accelerator architecture (top module taod_top). The block is driven over AXI4 by the VEGA host and by system memory.

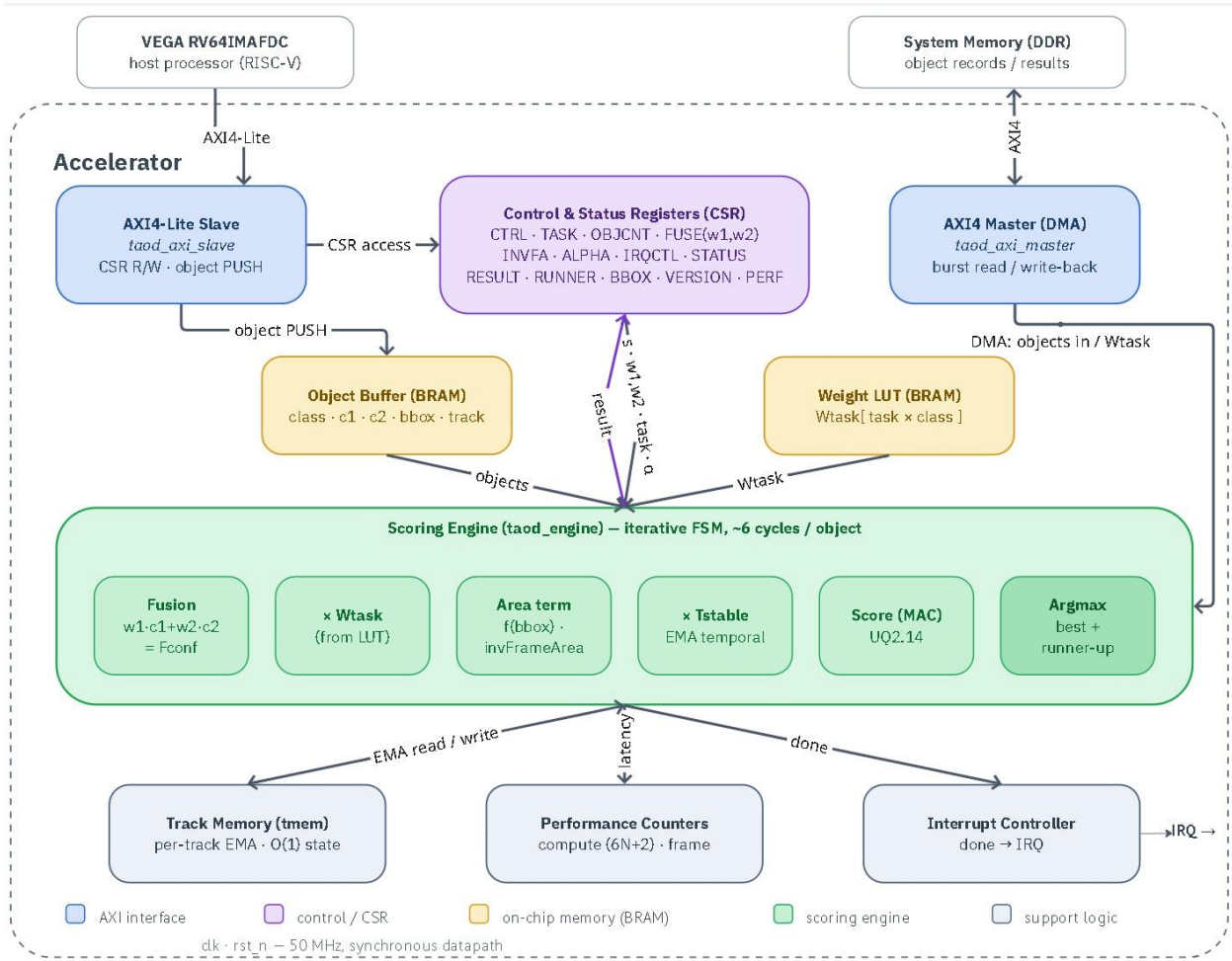


Figure 1 — TAOD accelerator design block diagram.

The accelerator accepts inputs through two paths. In CPU-push mode, the host writes configuration registers (task ID, fusion weights $w1/w2$, EMA coefficient, object count) and pushes object records through the AXI4-Lite slave interface. In DMA mode, the AXI4 master autonomously burst-reads object records from memory and writes results back. Object records are held in an on-chip object buffer (BRAM) and the per- (task, class) relevance weights in a weight LUT (BRAM).

The scoring engine is an iterative finite-state machine that processes one object every few cycles. Its datapath performs confidence fusion, the task-weight multiply, the bounding-box area term, the temporal-stability multiply (with per-track EMA state held in the track memory), and the final score, followed by an argmax that maintains the best object and the runner-up. The result is written to the RESULT/RUNNER/BBOX status registers for host readback (and optionally back to memory via the master). Two performance counters expose the measured cycle counts, and an interrupt is raised on completion.

3. Simulation Setup Block Diagram

Figure 2 shows the verification environment. The testbench (taod_tb) is self-checking: every result produced by the RTL is compared against an independent golden reference model.:

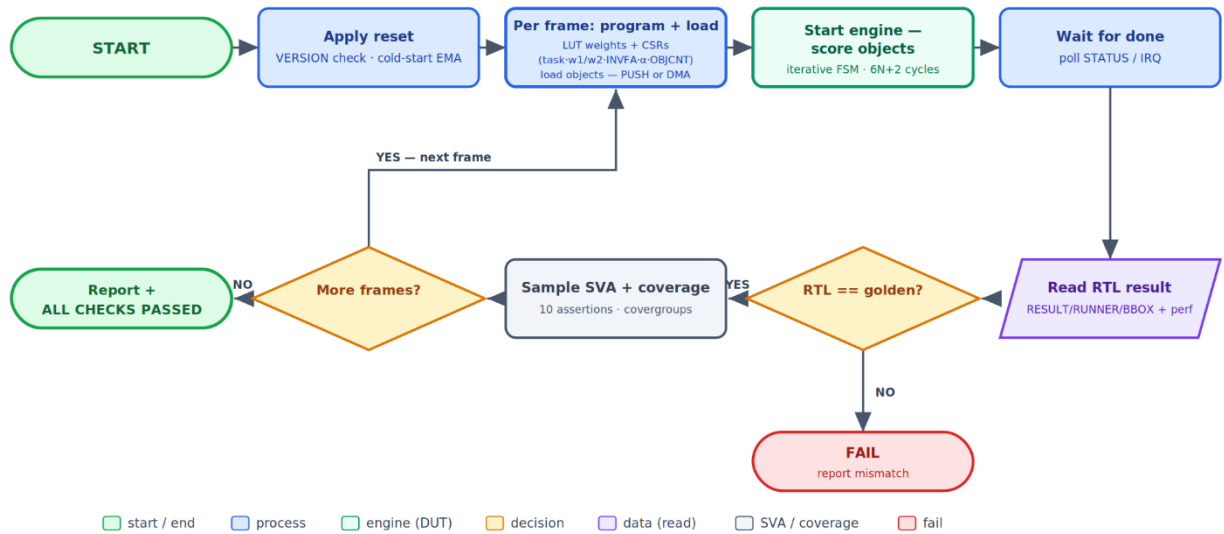


Figure 2 — Simulation setup / verification environment.

The stimulus sequencer drives a fixed campaign of frames (eight CPU-push frames and two DMA frames) covering different task IDs, object counts, fusion weights, valid/invalid objects, and an empty frame. The same stimulus is applied to the golden reference model, which computes the expected score, winning object, and runner-up in software. On the master port an AXI4 memory model (DDR BFM) serves object records for the DMA frames and captures the result write-back.

The scoreboard compares the RTL output against the golden model through both the result registers and the DDR write-back. In parallel, ten SystemVerilog assertions monitor protocol and behavioral properties, and a functional-coverage model (covergroups plus a manual bin tally) records which scenarios were exercised. A run ends with a pass/fail verdict and a coverage report.

4. Summary of RTL

The design comprises five synthesizable RTL files plus three verification files. All arithmetic uses UQ2.14 fixed-point (unity = 16384). The compile order is package-first; every file imports the package.

File	Role
taod_pkg.sv	Parameters, data types, address-map constants, fixed-point format.
taod_engine.sv	Scoring engine: iterative FSM and data path (fusion, task-weight, area term, EMA temporal term, score, argmax with runner-up).
taod_axi_slave.sv	AXI4-Lite slave: register read/write, object push, address decode.
taod_axi_master.sv	AXI4 master: DMA burst-read of object records and result write-back.
taod_top.sv	Top level: integrates engine, AXI interfaces, control/status registers, performance counters, interrupt.

4.1 Key design characteristics

- Iterative engine: one object is processed roughly every six cycles, trading throughput for a very small area footprint.
- Temporal-stability term: a per-track exponential moving average ($O(1)$ state per track) stabilizes selection across frames — a feature absent from the single-image reference formulation.
- Table-driven relevance: task-to-class relevance is held in a reconfigurable on-chip LUT, so the policy can be changed without modifying the RTL.
- Clean coding style: separated combinational and sequential logic, synchronous datapath, no inferred latches; all pipeline registers are reset for deterministic power-up.
- Two input paths: CPU-push (AXI4-Lite) and autonomous DMA (AXI4 master), producing identical results.
- Self-measuring: control/status registers expose two cycle counters (compute and full-frame) for on-chip latency measurement.

5. Simulation Results

The testbench runs ten frames (eight CPU-push, two DMA). For every frame the RTL result is checked against the golden model, all assertions are evaluated, and functional coverage is sampled. The run completes with:

RESULT: ALL CHECKS PASSED

(golden == RTL: 8 PUSH + 2 DMA frames, SVA clean)

The design was cross verified on multiple simulators (Icarus Verilog and QuestaSim; the source is also portable to Xcelium). Coverage measured in QuestaSim is summarized below.

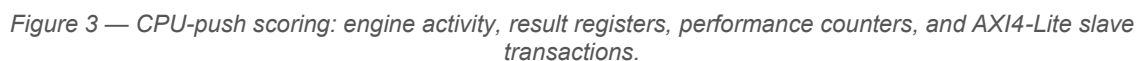
Coverage metric	Result	Notes
Assertions (SVA)	10/10 (100%)	0 failures; all directives hit.
FSM state	100%	Every engine / AXI state reached.
Statement	~97%	Uncovered lines are unreachable error branches.
Functional covergroups	~91%	Per-task and task × area cross bins.

Functional waveforms confirm correct operation.

Figure 3 shows a CPU-push scoring frame: the engine launches (eng_start), stays busy while it iterates over the object records (obj_addr stepping), and produces a settled score with the winning class and index, the runner up score is tracked in parallel.

The performance counters (perf_comp_last / perf_frame_last) report the measured cycle count, and the AXI Lite handshakes complete clean.

The VERSION register reads back its fixed identifier 0x7A0D_0001, confirming the read path end-to-end.



Timing diagram for AXI Master (DMA) showing signals like aoad_tbit_dutlm_arvalid, aoad_tbit_dutlm_arready, aoad_tbit_dutlm_araddr, aoad_tbit_dutlm_rvalid, aoad_tbit_dutlm_rdata, aoad_tbit_dutlm_arinvalid, aoad_tbit_dutlm_arready, aoad_tbit_dutlm_rinvalid, aoad_tbit_dutlm_rdata, aoad_tbit_dutdma_busy, aoad_tbit_dutdma_done, and aoad_tbit_dutlrmq. The diagram shows a sequence of transactions over time, with a cursor at 975074 ps.

Figure 4 — DMA mode: AXI4 master burst read of object records and result write-back.

6. Current Status

The RTL is complete verified, and has been synthesized for the target device (Xilinx XC7K325T). Post-synthesis resource utilization is very low:

Resource	Used	Device %
Slice LUTs	1,235	0.61%
Slice registers (FF)	1,164	0.29%
DSP48E1	10	1.19%
Block RAM (RAMB36)	6	1.35%

Timing closes comfortably at the 50 MHz target, with a worst negative slack of +10.669 ns and an implied maximum frequency of roughly 107 MHz — over a 2× margin. The low utilization and timing headroom make this an efficient, low-power design point.

Note on I/O utilization: when the accelerator is synthesized standalone as the top level, every AXI signal becomes a physical pin and the reported I/O count is high. This is an artifact of block-level synthesis; at SoC integration the AXI ports connect to the on-chip interconnect rather than pins. Synthesizing out-of-context yields representative figures and is the appropriate methodology for an integratable block.

6.1 Stage 2B refinements

- Synchronous-reset cleanup: all engine pipeline registers are now reset, removing a synthesis warning and any simulation-vs-silicon mismatch risk at power-up.
- On chip performance counters are added (control/status offsets 0x120 and 0x128) for direct latency measurement.
- A real AXI master write-channel ordering bug (WVALID/WLAST alignment) found and fixed in an earlier stage; the master now meets AXI write-handshake requirements.

6.2 Remaining work (Stage 3)

- SoC integration with the VEGA processor on the Genesys-2 board.
- Host application that feeds dataset images through the detection pipeline to the accelerator.
- On-board measurement of latency and power, and end-to-end accuracy on the task test cases.

7. Acceleration Achieved

The engine's latency was measured on-chip using the performance counters and follows an exact linear relation in the number of objects N :

$$\text{Engine latency} = 6 \times N + 2 \text{ clock cycles (at 50 MHz / 20 ns per cycle)}$$

This was confirmed against the simulation for several object counts:

Objects N	Cycles ($6N+2$)	Time @ 50 MHz
1	8	0.16 μ s
2	14	0.28 μ s
3	20	0.40 μ s
10	62	1.24 μ s
64 (max)	386	7.72 μ s

Per object this is six cycles (120 ns), with the DMA frame counter additionally including the object-read time.

For comparison, performing the same per-object scoring in software on the VEGA core (RV64IMAFDC, with the M extension providing hardware integer multiply, at 50 MHz) requires on the order of 25–30 RISC-V instructions per object — confidence fusion, the LUT index and lookup, the task-weight and area-term multiplies, the EMA load/update/store, and the argmax compare. On a single-issue in-order core, allowing for multi-cycle multiplies and load latency, this is roughly 50–90 cycles per object, versus six cycles in the accelerator: approximately an order-of-magnitude speedup (about 8–15 $\times$), before accounting for the per-object loop overhead the hardware avoids and the overlap of DMA fetch with computation.

Honest basis: the accelerator figure ($6N+2$ cycles) is measured; the software figure is an instruction-count estimate. The exact software baseline will be confirmed on VEGA at Stage 3 using the RISC-V minstret / mcycle counters.

Summary: a verified, efficient task-aware object-selection accelerator — ~1% device utilization, timing closed at 50 MHz with >2 $\times$ margin, 100% assertion and FSM-state coverage, ~91% functional coverage, and a measured $6N+2$ -cycle selection latency.